Ответы на контрольные вопросы ПР6 (№ 12–13)

Индивидуальный проект «Сервис планирования киносъёмок» (веб-интерфейс «КиноПлан»)

Лимонов Никита Андреевич, ЭФБО-08-24

В моей предметной области роль «комнаты» играет **локация** (место съёмок с ограниченной вместимостью), а роль «бронирования» — **съёмка** (локация, занятая под сцену фильма в определённую дату).

12. Передача данных во включаемый шаблон

Какие данные используются в карточке. Строка-карточка локации вынесена в `locations/includes/location_row.html`:

```
<tr>
    <td>
        <a href="{% url 'locations:detail' location.id %}">
            {{ location.name|default:"Без названия" }}
        </a>
    </td>
    <td>{{ location.address|default:"адрес не указан" }}</td>
    <td>{{ location.capacity }}</td>
</tr>
```

Фрагмент использует одну переменную `location` и четыре её поля: `id` (для ссылки), `name`, `address`, `capacity`. Своих данных у фрагмента нет — он берёт их из контекста шаблона, который его подключил.

Откуда берутся данные. Цепочка такая: SQL-запрос → представление → шаблон списка → цикл → включаемый шаблон.

```
# db/queries.py — имена столбцов станут именами полей
SELECT id, name, address, capacity FROM locations WHERE capacity >= ?
...

# locations/views.py — ключ контекста "locations"
context = {"locations": queries.get_locations(min_capacity or 0), ...}

# locations/location_list.html — переменная цикла "location"
{% for location in locations %}
    {% include "locations/includes/location_row.html" %}
{% endfor %}
```

`{% include %}` по умолчанию передаёт фрагменту **весь текущий контекст**, включая переменную цикла. Поэтому имя `location` в цикле должно совпадать с именем внутри фрагмента. Тот же фрагмент подключается на странице свободных локаций (`free_locations.html`) — там цикл тоже называется `location`.

Эксперимент 1: переименовываю переменную цикла. В `location_list.html` меняю `{% for location in locations %}` на `{% for loc in locations %}`, фрагмент не трогаю.

Вариант	Результат рендеринга
Исходный (`location`)	`<a href="/locations/2/">Студия «Север»</a>` | пр. Мира, 119 | 80
Цикл `loc`, без ссылки во фрагменте	Без названия | адрес не указан | *(пусто)*
Цикл `loc`, фрагмент как есть	`NoReverseMatch: Reverse for 'detail' with arguments '('',)' not found` — страница падает с ошибкой

Почему так: внутри фрагмента ищется `location`, а в контексте теперь есть только `loc`. Django **не выдаёт ошибку на неизвестную переменную** — он молча подставляет пустую строку. Поэтому:

- `{{ location.name|default:"Без названия" }}` показывает значение по умолчанию — кажется, что у локации нет названия, хотя в базе оно есть;
- `{{ location.capacity }}` без `default` выводит пустую ячейку;
- `{% url 'locations:detail' location.id %}` получает пустую строку вместо числа, а маршрут ожидает `<int:location_id>`, — адрес построить нельзя, и Django выбрасывает `NoReverseMatch`.

Эксперимент 2: меняю имя поля во фрагменте. Пишу `{{ location.title }}` вместо `{{ location.name }}`. Результат: «Без названия | 80». Поля `title` нет — запрос выбирает столбец `name`, а у строки `sqlite3.Row` есть только ключи `['id', 'name', 'address', 'capacity']`. Ошибки снова нет, данные просто пропадают. То же случится, если в SQL переименовать столбец псевдонимом (`name AS location_name`), а во фрагменте оставить `location.name`.

Как передавать данные надёжно. Явно указать, что получает фрагмент, через `with`:

```
{% for loc in locations %}
    {% include "locations/includes/location_row.html" with location=loc
%}
{% endfor %}
```

С `with location=loc` строка снова выводится полностью (`/locations/2/`, «Студия «Север»», 80). Если добавить `only`, фрагмент получит **только** перечисленные переменные — тогда его зависимость от контекста видна прямо в месте подключения.

Вывод: данные перестают отображаться, когда имя переменной (или поля) во фрагменте не совпадает с тем, что реально лежит в контексте. Шаблонизатор Django в такой ситуации не сообщает об ошибке, а выводит пустую строку (или значение `default`), поэтому поломку видно только по странице. Исключение — `{% url %}`: без корректного аргумента он падает с `NoReverseMatch`.

13. Формирование динамических URL

Маршруты с идентификатором. У страницы локации и страницы съёмки в адресе есть целочисленный параметр:

```
# locations/urls.py                          # shootings/urls.py
app_name = "locations"                       app_name = "shootings"
path("<int:location_id>/",                   path("<int:shooting_id>/",
     views.location_detail,                       views.shooting_detail,
     name="detail"),                              name="detail"),
```

Ссылки в шаблонах. Адрес не пишется вручную, его строит тег `{% url %}` по имени маршрута `"пространство_имён:имя"` и идентификатору записи:

```
<!-- locations/includes/location_row.html — ссылка на локацию
(«комнату») -->
<a href="{% url 'locations:detail' location.id %}">{{ location.name }}
</a>

<!-- shootings/includes/shooting_item.html — ссылка на съёмку
(«бронирование») -->
<a href="{% url 'shootings:detail' shooting.id %}" ...>
```

Django находит маршрут `detail` в приложении `locations`, берёт его шаблон `locations/<int:location_id>/` и подставляет переданное значение.

Эксперимент: меняю передаваемый идентификатор. Для локации «Вокзал, павильон 3» (`id = 1`, `capacity = 40`) и съёмки № 5:

Что передано в `{% url %}`	Получившийся адрес	Ответ сервера
`'locations:detail' location.id`	`/locations/1/`	200 — «Вокзал, павильон 3»
`'locations:detail' 3`	`/locations/3/`	200 — «Квартира на Арбате»
`'locations:detail' location.capacity`	`/locations/40/`	404 — локации с id 40 нет
`'locations:detail' 'abc'`	—	`NoReverseMatch` при рендеринге
`'shootings:detail' shooting.id` (id = 5)	`/shootings/5/`	200 — съёмка 05.11.2026
`'shootings:detail' 2`	`/shootings/2/`	200 — отменённая съёмка 22.10.2026

Что видно из эксперимента:

- Меняется только последняя часть адреса — та, что соответствует `<int:location_id>`. Начало `/locations/` берётся из `kinoplan/urls.py` (`path("locations/",`

`include("locations.urls"))`).

- Если передать не то поле (`capacity` вместо `id`), адрес строится без ошибки, но ведёт на другую запись или на 404. Тег проверяет только тип значения, а не то, что это правильный идентификатор.
- Если значение не подходит под конвертер `int` (строка `'abc'` или пустое значение), Django не может построить адрес и выдаёт `NoReverseMatch`. Набранный вручную `/locations/abc/` не совпадает ни с одним маршрутом — 404.

Параметры можно передавать и по имени, результат тот же: `{% url 'locations:detail' location_id=location.id %}` → `/locations/1/`. GET-параметры дописываются после тега: `{% url 'locations:detail' location.id %}?date=2026-11-10` → `/locations/1/?date=2026-11-10` (проверка занятости локации на дату).

Почему не писать адрес вручную (`href="/locations/{{ location.id }}/"`): если в `urls.py` поменять `"locations/"` на `"places/"`, все ссылки через `{% url %}` обновятся сами, а ручные адреса придётся искать и править в каждом шаблоне.